systheme de gestions d’archives

cahier conceptuelle

—

Membres de l'équipe technique :

1. FOSSONG TSOFACK PATRICIA - *Matricule : 24G2975*
2. MELI WILLIAM - *Matricule : 2512257*
3. SINENG KENGNI JUVENAL - *Matricule : 24H2194*
4. NOE FOGAING DARCHEVY - *Matricule : 23U2566*
5. TCHINDA FOGANG PIERRE LEGRAND - *Matricule : 24H2001*
6. NGHOMSI FEUKOUO RAVEL - *Matricule : 23V2011*
7. WOGUE DEFFO JEAN MARC - *Matricule : 24H2428*

chef de projet : ***MR. LONCHI George***

Vue d'ensemble

Avec l'augmentation du volume de documents dans les entreprises et institutions, il est devenu essentiel de disposer d'un système efficace pour scanner, archiver, organiser et consulter les documents. L'objectif de cette application est de fournir une solution intuitive et sécurisée permettant :

- La numérisation et l'archivage des documents papier.
- La consultation rapide et la recherche de documents.
- La gestion sécurisée des utilisateurs et des actions sur les documents.

L'application se concentre sur les besoins essentiels pour une gestion efficace des documents et est évolutive pour intégrer des fonctionnalités futures telles que le partage et la collaboration.

Objectifs

La mise en place de la plateforme ScanVault vise à atteindre les objectifs suivants :

1. Scanner et archiver tous les documents papier ou numériques.
2. Organiser les documents avec un système de dossiers.
3. Permettre la consultation, la recherche et le suivi des documents.
4. Assurer la sécurité et la confidentialité des documents.
5. Fournir une interface simple et intuitive pour tous les utilisateurs.

Besoins de l'application :

Afin de répondre aux attentes des utilisateurs, l'application intègre les modules fonctionnels suivants :

besoins fonctionnels :

- Scanner et enregistrer un document.
- lire, telecharger, deplacer ou supprimer les documents
- Importer un document existant (PDF, image).
- Créer, editer, supprimer des dossiers pour organiser les documents.
- rendre un dossier public ou privé
- Rechercher et filtrer les documents par nom, date de creation ou code.
- creer, supprimer , editer un espace de travail
- partager un espace de travail avec d'autres personnes (profiles)
- Consulter l'historique des actions (qui a scanné ou modifié le document).
- Authentification (creation de co;pte utilisteur , login, logout, suppression de compte)

besoins non fonctionnels :

- Performance : Scan et sauvegarde en moins de 5 secondes.
- Sécurité : Authentification obligatoire, contrôle d'accès selon rôle.
- Fiabilité : Fonctionnement stable avec messages d'erreur clairs; persistence de la base de donnee
- Ergonomie : Interface simple et intuitive.
- Compatibilité : Fonctionne sur android et ios, compatible avec différents modèles de scanner.

Diagrammes:

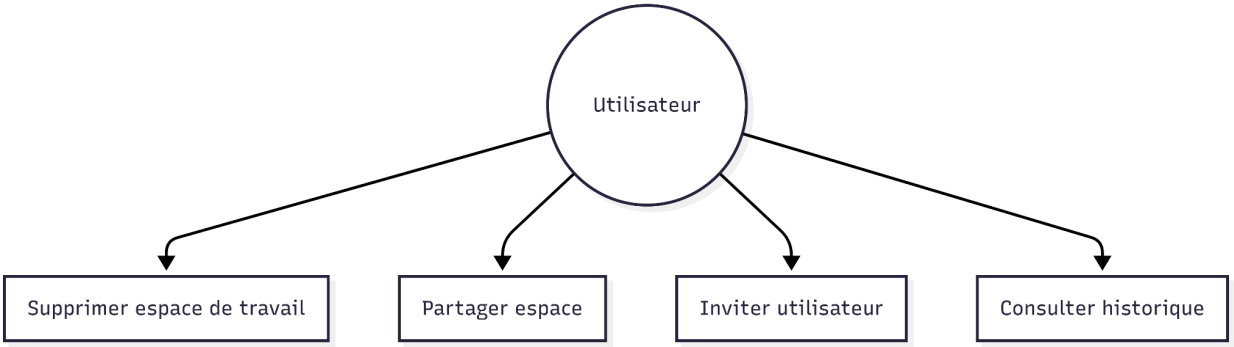
Diagrammes de cas d'utilisations :

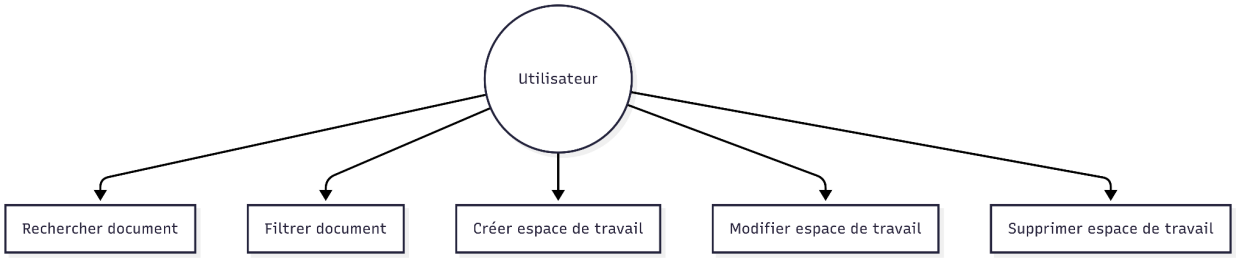
Les acteurs principaux sont l'**Utilisateur (Standard)** et le **Système**. Les actions s'articulent autour des axes majeurs :

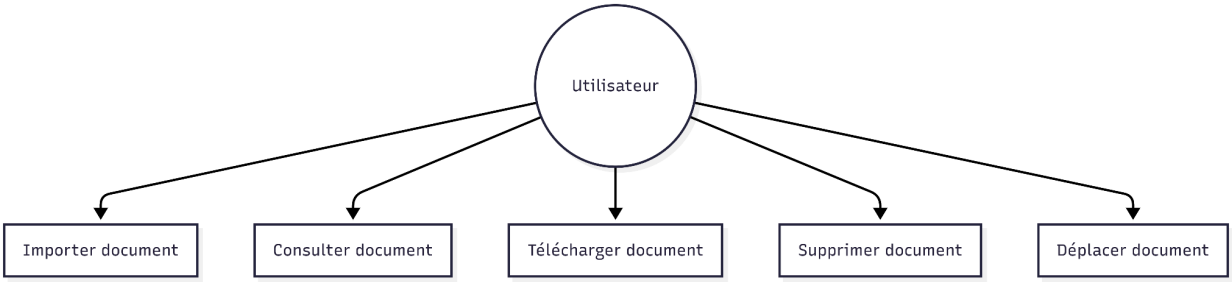
Cas principaux :

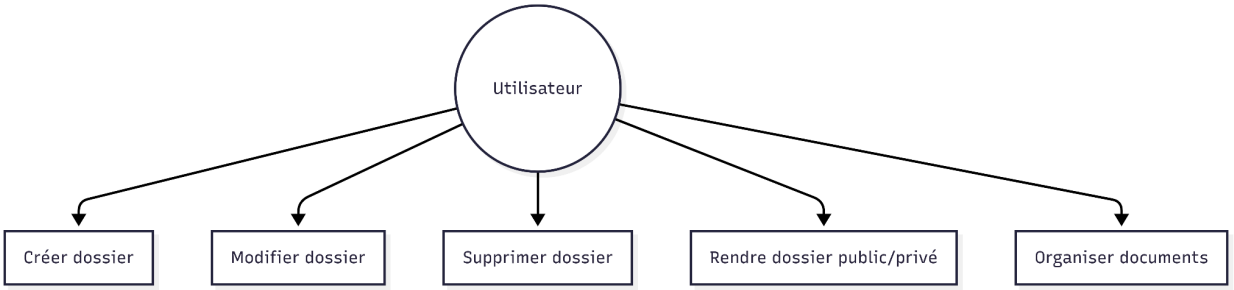
- Scanner / Importer document
- Organiser documents (créer, renommer, supprimer dossiers)

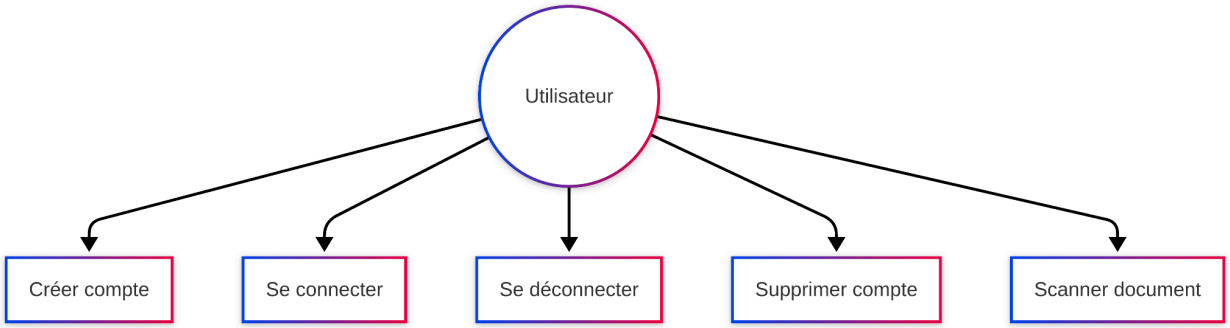
- Rechercher document
- Rendre un dossier public ou prives
- Creer un espace
- Creer un dossier
- Suppri;er un dossier
- Inviter un profil sur un espace ou dossier
- Consulter historique











Diagrammes de classes :

Le système est modélisé autour de 4 classes métiers fondamentales :

- **Utilisateur :** Est lié à des `Espaces`, des `Dossiers` et déclenche des `Historiques`.
- **Espace :** Agrège 0 à plusieurs `Dossiers`.
- **Dossier :** Contient 0 à plusieurs `Documents`. Possède l'attribut booléen `isPublic`.
- **Document :** Appartient à 1 `Dossier`. Possède des attributs de métadonnées (nom, type, status, path, isDeleted).
- **Historique :** Relie 1 `Utilisateur` et 1 `Document` à un instant T (`actionDate`) avec un type d'action donné (`actionType`).

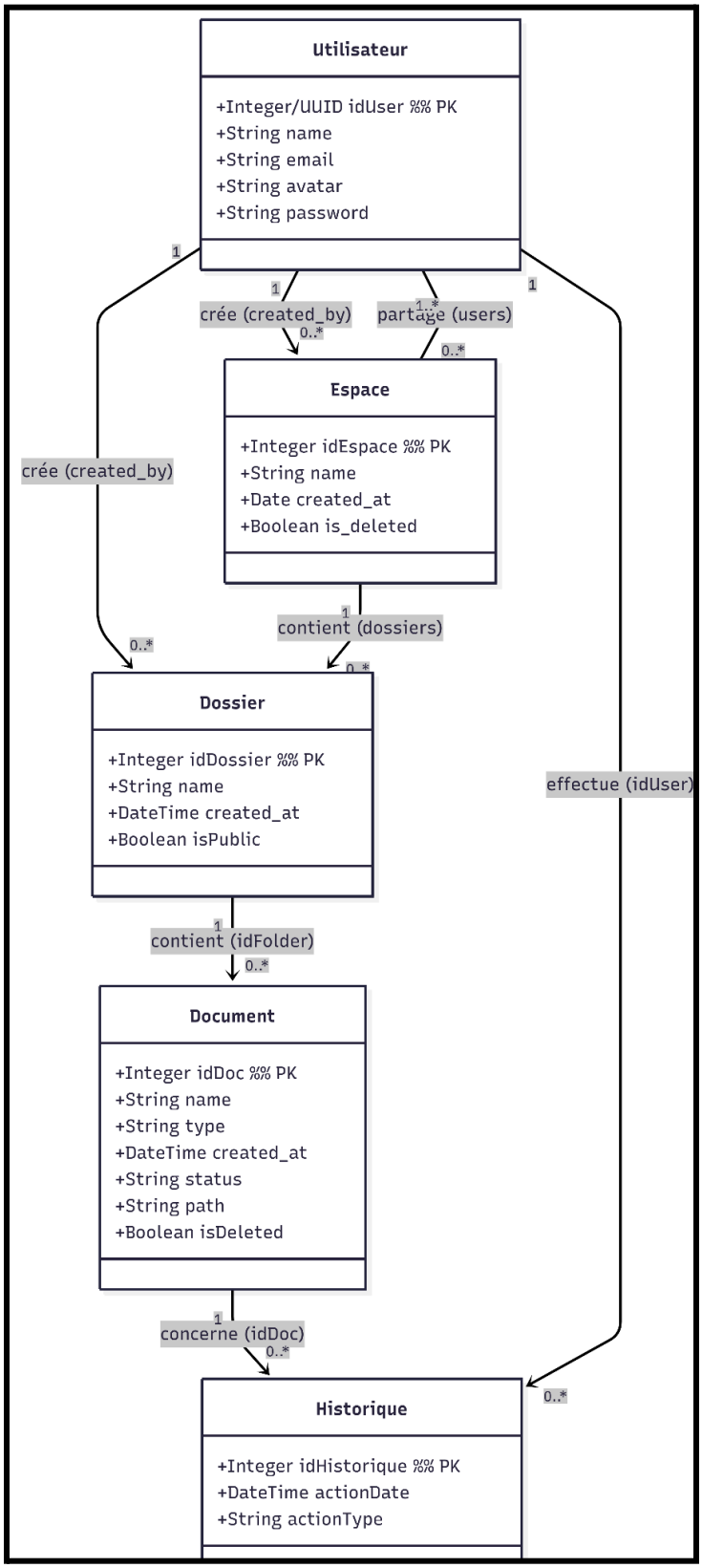
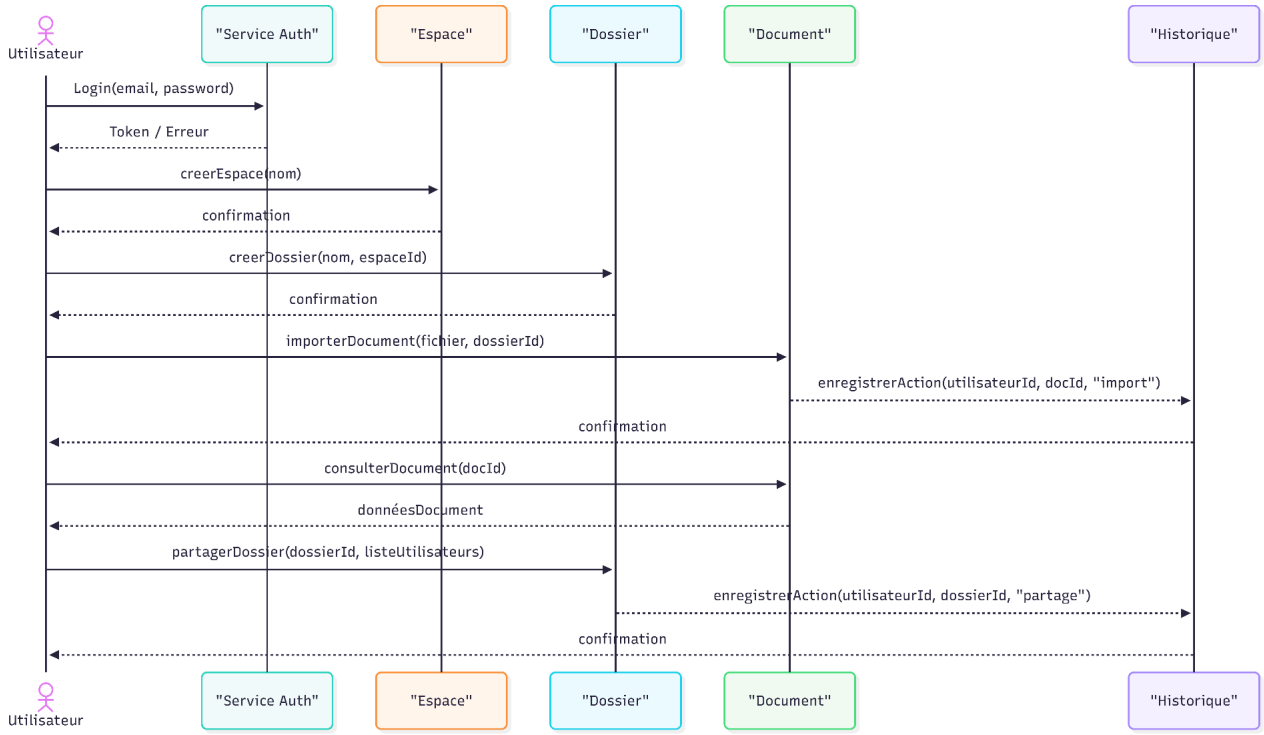

Utilisateur
+Integer/UUID idUser %% PK
+String name
+String email
+String avatar
+String password
1
1
crée (created_by)
0..*
1..*
partage (users)
0..*
1
Espace
+Integer idEspace %% PK
+String name
+Date created_at
+Boolean is_deleted
crée (created_by)
0..*
1
contient (dossiers)
0..*
Dossier
+Integer idDossier %% PK
+String name
+DateTime created_at
+Boolean isPublic
effectue (idUser)
1
contient (idFolder)
0..*
Document
+Integer idDoc %% PK
+String name
+String type
+DateTime created_at
+String status
+String path
+Boolean isDeleted
1
concerne (idDoc)
0..*
0..*
Historique
+Integer idHistorique %% PK
+DateTime actionDate
+String actionType

Diagramme de Séquence:

1. L'Utilisateur envoie ses identifiants au *Service Auth*.
2. Le Service retourne un *Token* de session.
3. L'Utilisateur navigue dans un *Espace* et sélectionne un *Dossier*.
4. L'Utilisateur lance l'action `importerDocument(fichier, dossierId)` vers l'entité *Document*.
5. Le système enregistre l'entité, et déclenche de manière asynchrone `enregistrerAction(...)` vers le module *Historique*.
6. Une confirmation est renvoyée à l'utilisateur.



Modèle de la Base de données

L'architecture des données repose sur un schéma relationnel robuste (RDBMS) :

Table Utilisateur

- `idUser` (Primary Key, Integer/UUID) : Identifiant unique.

- `name` (String) : Nom complet.
- `email` (String) : Adresse électronique (Unique).
- `avatar` (String) : URL de la photo de profil.
- `password` (String) : Mot de passe (haché par le service Auth).

Table Espace

- `idEspace` (Primary Key, Integer) : Identifiant unique de l'espace.
- `users(FK->user): utilisateurs avec qui c'est partagés`
- `dossiers(FK->dossiers) : dossiers qui sont inclus dans l'espace`
- `name` (String) : Nom de l'espace de travail.
- created_at(date): date de creation du dossier
- created_by (FK->users):proprietaire
- is_deleted:si c'est supprimé ou pas

Table `Dossier`

- `idDossier` (Primary Key, Integer) : Identifiant unique.
- `name` (String) : Nom du dossier.
- `created_at` (DateTime) : Date de création.
- `created_by` (Foreign Key -> Utilisateur.idUser) : Créateur du dossier.
- `isPublic` (Boolean) : Détermine si le dossier est partagé ou privé.

Table `Document`

- `idDoc` (Primary Key, Integer) : Identifiant unique du fichier.
- `name` (String) : Nom du document.
- `type` (String) : Extension (pdf, jpg, png, etc.).
- `created_at` (DateTime) : Date d'importation/scan.
- `status` (String) : État actuel du document (actif, archivé).
- `path` (String) : Chemin d'accès ou URL de stockage (Bucket Cloud).
- `isDeleted` (Boolean) : Utilisé pour la suppression logique (Soft Delete), évite la perte accidentelle de données.
- `idFolder` (Foreign Key -> Dossier.idDossier) : Dossier parent.

Table `Historique`

I. `idHistorique` (Primary Key, Integer) : Identifiant de la trace.
II. `idDoc` (Foreign Key -> Document.idDoc) : Document concerné.
III. `idUser` (Foreign Key -> Utilisateur.idUser) : Auteur de l'action.
IV. `actionDate` (DateTime) : Horodatage précis de l'événement.

V. `actionType` (String) : Nature de l'action (Création, Modification, Importation, Partage, Suppression).

8. Technologies et Outils

Pour garantir modernité, sécurité et rapidité de développement, la pile technologique (Stack) retenue est la suivante :

- Frontend (Application Mobile / Web) :
  - **Flutter :** Framework idéal pour un déploiement multiplateforme (iOS et Android) avec une seule base de code.
  - *Alternative web/admin :* React ou Node.js (pour les interfaces de gestion spécifiques).
- Backend (BaaS - Backend as a Service) :
  - * **Supabase :** Fournit l'authentification native, les règles de sécurité row-level security (RLS), et le stockage (Buckets) pour les fichiers PDF/Images.
- Base de données :
  - * **PostgreSQL :** Base de données relationnelle extrêmement puissante, hébergée de manière transparente via Supabase.

Conclusion

cette application se positionne comme une application ambitieuse, pensée pour être à la fois simple d'utilisation, intuitive et hautement sécurisée. Elle a pour vocation de simplifier radicalement la manière dont les utilisateurs scannent, archivent, organisent et consultent leurs documents au quotidien.

Le projet se concentre de manière pragmatique sur la livraison d'un "Niveau 1" fonctionnel, répondant aux besoins essentiels de la gestion documentaire. Cependant, grâce à une architecture robuste basée sur Flutter et PostgreSQL (Supabase), le système est prêt à évoluer pour accueillir des fonctionnalités collaboratives de pointe à l'avenir, assurant ainsi la pérennité du projet.